La clase RandomAccessFile en Java

- Se usa para **leer y escribir en cualquier posición de un fichero**.
- Está en el paquete java.io y permite **acceso aleatorio**: no hace falta leer todo el fichero para llegar a una parte concreta.

Constructores

RandomAccessFile raf = new RandomAccessFile("archivo.dat", "r");

RandomAccessFile raf = new RandomAccessFile("archivo.dat", "rw");

- "r" → modo **solo lectura**.
- "rw" → modo **lectura y escritura**. Si el fichero no existe, lo crea.

Métodos principales

1. Posicionamiento en el fichero

- **seek(long pos)**→ mueve el puntero de lectura/escritura a la posición pos (en bytes, desde el inicio).
    - raf.seek(0); // ir al inicio
    - raf.seek(raf.length()); // ir al final para añadir
- **getFilePointer()**→ devuelve la posición actual del puntero.

```
long pos = raf.getFilePointer();
System.out.println("Estoy en el byte: " + pos);
```

- **length()** → devuelve el tamaño del fichero en bytes.
    - System.out.println("Tamaño del fichero: " + raf.length());
- **setLength(long newLength)**→ cambia el tamaño del fichero.
    - Si newLength < length(), **trunca** (borra bytes del final).
    - Si newLength > length(), añade bytes vacíos al final.

2. Escritura

Escribe datos **en binario**, no como texto plano.

- **Tipos primitivos**:
    - writeInt(int v)
    - writeDouble(double v)
    - writeBoolean(boolean v)
    - writeChar(int v)
    - writeUTF(String s) (escribe longitud + caracteres en UTF-8)

Los métodos writeInt(), writeDouble(), writeUTF()... **no escriben caracteres legibles**, sino **bytes binarios**.

- Ejemplo: raf.writeInt(123) no pone "123" en el fichero, sino 4 bytes en binario que representan ese número.
- Si abres el fichero con un editor de texto, verás símbolos raros.

writeUTF() sí escribe cadenas, pero **les añade información extra** (longitud).

- O sea, "Pedro" no se guarda como P e d r o, sino como [2 bytes longitud][bytes de cada carácter].

Por eso, RandomAccessFile es útil cuando trabajas con **registros fijos o binarios**, como una **mini base de datos en disco**.

Ejemplo:

```
raf.writeInt(123);
raf.writeUTF("Hola");
raf.writeDouble(9.75);
```

Método	Qué escribe	Tamaño bytes	Legible en Bloc de notas
writeInt(123)	Número 123 en binario fijo	4 bytes	No
writeUTF("123")	Longitud (2 bytes) + caracteres 1 2 3	5 bytes	Parcial (verás 123, pero hay bytes ocultos delante)
FileWriter.write("123")	Solo los caracteres	3 bytes	Sí

- writeInt guarda un **número** como dato binario, no como texto.
- writeUTF guarda una **cadena** en binario, añadiendo su longitud.
- FileWriter guarda **texto plano**, lo que se puede abrir y leer tal cual

Los bytes que ocupa cada tipo dato:

Tipo	Tamaño en bytes	Bits	Comentario
byte	1	8	Entero con signo (-128 a 127)
short	2	16	Entero con signo
char	2	16	Carácter Unicode (UTF-16)
int	4	32	Entero con signo
long	8	64	Entero con signo
float	4	32	Decimal precisión simple (IEEE 754)
double	8	64	Decimal precisión doble (IEEE 754)
boolean	1*	—	En RAM no está estandarizado, pero en RandomAccessFile.writeBoolean ocupa 1 byte (0 o 1)

3. Lectura

Debes leer en el **mismo orden y tipo** en que escribiste.

- readInt()
- readDouble()
- readBoolean()
- readChar()
- readUTF()

Ejemplo:

```
int num = raf.readInt();
String txt = raf.readUTF();
double nota = raf.readDouble();
```

Si los lees en distinto orden, saldrán datos corruptos.

4. Cierre

- **close()** → cierra el fichero y libera recursos.
    - Si usas *try-with-resources*, no necesitas llamarlo manualmente.

¿Cuándo sí usar RandomAccessFile?

Si quieres guardar **registros binarios** y poder ir **directamente a uno de ellos sin leer todo el fichero**.

Ejemplo clásico:

- Guardar empleados: int id, String nombre, double salario.
- Luego saltar al 3er empleado con seek(posicion) y leer solo ese registro.

Es como una **base de datos primitiva en fichero**, no un fichero de texto.